

IFJ - dokumentace k projektu

Vojtěch Voldřich	(xvoldrv00)	25 %
Jakub Jambrozi	(xjambrj00)	25 %
Petr Kotek	(xkotekp00)	25 %
Josef Pasek	(xpasekj00)	25 %

Tým xvoldrv00, Varianta TRP

3. prosince 2025

Implementovaná rozšíření

FUNEXP
EXTSTAT
EXTFUN
BOOLTHEN
CYCLES
OPERATORS
ONELINEBLOCK
STATICAN

Obsah

1 Lexikální analýza **3**

2 Syntaktická analýza **5**
2.1 Překladač (Parser) 5
2.2 Precedenční analýza (PSA) 7

3 Sémantická analýza **8**

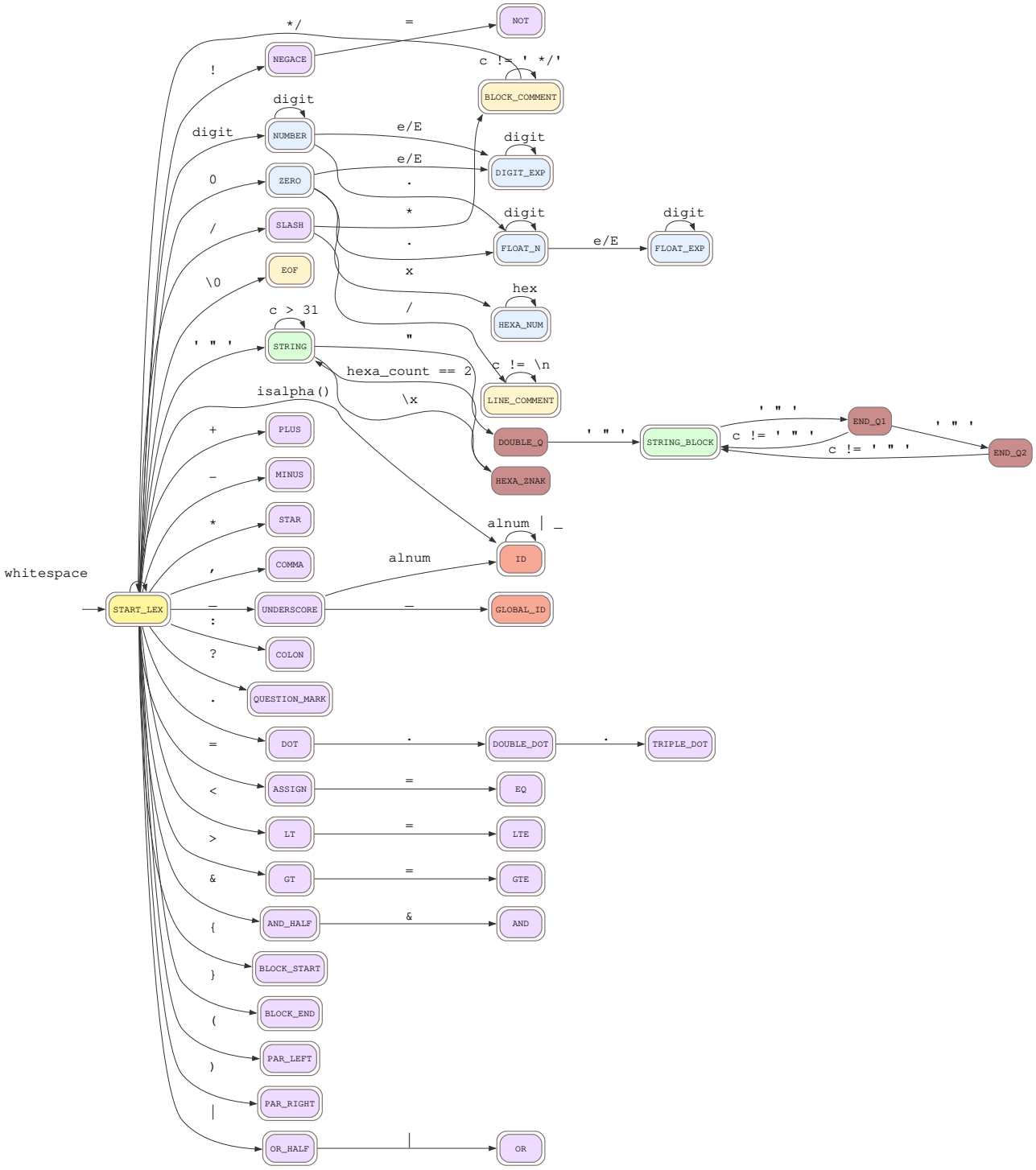
4 Generování kódu **9**
4.1 Obecné principy 9
4.2 Správa návěští 9
4.3 Práce s proměnnými a paměťové rámce 9
4.4 Volací konvence 9
4.5 Vestavěné funkce 10

5 Datové struktury **11**
5.1 Tabulka symbolů 11
5.2 Generický dynamický zásobník 11
5.3 Abstraktní syntaktický strom 11

6 Způsob práce v týmu **12**
6.1 Rozdělení práce v týmu 12
6.2 Spolupráce 12
6.2.1 Průběh vývoje 12

Lexikální analýza

Lexikální analýza je implementována pomocí deterministického konečného automatu (DKA). Většina tokenů je získána přímo ze vstupního řetězce pomocí funkce `substring`. Výjimku tvoří token `TOKEN_STRING`, u kterého se obsah sestavuje průběžně pomocí funkce `appendChar`. Tento přístup je nezbytný pro správné zpracování escape sekvencí a převod hexadecimálních čísel na odpovídající znaky. V okamžiku nalezení platného tokenu přechází automat na návěští `token_found` nebo na návěsti `token_found_string`, kde je zajištěno korektní uvolnění dočasně alokované paměti (bufferu) a vrácení výsledného tokenu.



Obrázek 1: Diagram deterministického konečného automatu pro lexikální analýzu

Syntaktická analýza

Překladač (Parser)

Parser je implementován v souborech `parser.h a parser.c` na základě LL(1) gramatiky, která je uvedena níže. Jedná se o prediktivní top-down parser, který zpracovává vstup postupným čtením tokenů volaním funkce `lexerTokenNext` z lexikálního analyzátoru.

Při zpracování každého neterminálu parser vybírá vhodné derivační pravidlo (např. `programFunc`, `prologFunc`, `varFunc`, ...) podle aktuálního symbolu na vrcholu zásobníku a podle aktuálně načteného tokenu. Volba produkčního pravidla je jednoznačná, což zaručují vlastnosti LL(1) gramatiky. Každá implementovaná funkce tak přímo reprezentuje jedno pravidlo z této gramatiky.

V případě nesouladu následujícího tokenu s pravidlem gramatiky program končí s chybovým kódem syntaktické analýzy (2).

Pro potřeby syntaktické analýzy byla navržena pomocná datová struktura `Symbol` v souboru `symbol.h`, která reprezentuje terminály i neterminály v gramatice. Každý symbol obsahuje informace potřebné pro syntaktické rozhodování (typ symbolu, případnou hodnotu tokenu a identifikátor terminálu nebo neterminálu).

Startovacím neterminálem syntaktické analýzy je `<program>`.

Kromě samotného rozpoznání syntaktické struktury parser zároveň vytváří i abstraktní syntaktický strom (AST). Ten je konstruován pomocí zásobníku uzlů `astStack`, do kterého se postupně vkládají listové i vnitřní uzly. Terminály, jejichž hodnotu je potřeba uchovat (identifikátory, literály), se ukládají do AST jako listové uzly pomocí funkce `newLeaf`.

Vnitřní uzly vznikají při aplikaci redukcí – každé derivační pravidlo má v implementaci přiřazeno odpovídající typ AST uzlu a aritu. Po provedení redukce parser odebere z `astStack` potřebný počet potomků, vytvoří nový AST uzel pomocí `astNew` a vloží jej zpět na vrchol zásobníku.

Tímto způsobem je AST sestavován současně s derivací a po zpracování všech pravidel tvoří kořen stromu jediný uzel, který reprezentuje celý program.

```
<program> →              <optNewlines> <prolog> TOKEN_NEW_LINE <optNewlines> <mainClass>
<prolog> →               TOKEN_KEYWORD_IMPORT <optNewlines> TOKEN_STRING("ifj25")
                         TOKEN_KEYWORD_FOR <optNewlines> TOKEN_KEYWORD_IFJ
<mainClass> →            TOKEN_KEYWORD_CLASS TOKEN_IDENTIFIER(Program) TOKEN_BLOCK_START
                         TOKEN_NEW_LINE <mainClassBlock> TOKEN_BLOCK_END <newLineCheck>
<mainClassBlock> →       <funcDec> <mainClassBlock> | TOKEN_NEW_LINE <mainClassBlock> |
                         eps
<newLineCheck> →         TOKEN_NEW_LINE <newLineCheck> | TOKEN_EOF
<optNewlines> →          TOKEN_NEW_LINE <optNewlines> | eps
<block> →                TOKEN_BLOCK_START <blockContentCheck> TOKEN_BLOCK_END
<blockContentCheck> →    <expr> | TOKEN_NEW_LINE <blockContent> | eps
<blockContent> →         TOKEN_NEW_LINE <blockContent> | <stmt> | eps
<stmt> →                 <stmtList> <stmtTail>
<stmtTail> →             TOKEN_NEW_LINE <blockContent> | eps
<stmtList> →             <return> | <var> | <if> | <while> | <for> | <break> | <continue>
                         | <callAssign> | <globalAssign> | TOKEN_IDENTIFIER <expr> |
                         <expr> | <block>
<funcDec> →              TOKEN_KEYWORD_STATIC TOKEN_IDENTIFIER <funcDecTail>
<funcDecTail> →          TOKEN_LPAREN <optNewlines> <param> TOKEN_RPAREN <block>
                         TOKEN_NEW_LINE | TOKEN_ASSIGN TOKEN_LPAREN TOKEN_IDENTIFIER
                         TOKEN_RPAREN <block> TOKEN_NEW_LINE | <block> TOKEN_NEW_LINE
<param> →                TOKEN_IDENTIFIER <paramList> | eps
<paramList> →            TOKEN_COMMA <optNewlines> TOKEN_IDENTIFIER <paramList> | eps
<var> →                  TOKEN_KEYWORD_VAR TOKEN_IDENTIFIER <varExt>
<varExt> →               TOKEN_ASSIGN <assignIfjIdCheck> | TOKEN_NEW_LINE |
                         TOKEN_BLOCK_END
```

<varArg> →	<expr> <varArgList> | eps
<varArgList> →	TOKEN_COMMA <optNewlines> <expr> <varArgList> | eps
<callAssign> →	TOKEN_IDENTIFIER <callAssignList>
<callAssignList> →	TOKEN_ASSIGN <assignIfjIdCheck> | TOKEN_LPAREN <varArg> TOKEN_RPAREN
<globalAssign> →	TOKEN_GLOBAL_ID TOKEN_ASSIGN <assignIfjIdCheck>
<if> →	TOKEN_KEYWORD_IF TOKEN_LPAREN <expr> TOKEN_RPAREN <block> <elseChain>
<elseChain> →	TOKEN_KEYWORD_ELSE <elseItem> <elseChain> | eps
<elseItem> →	TOKEN_KEYWORD_IF TOKEN_LPAREN <expr> TOKEN_RPAREN <block> | <block>
<while> →	TOKEN_KEYWORD_WHILE TOKEN_LPAREN <expr> TOKEN_RPAREN <block>
<for> →	TOKEN_KEYWORD_FOR TOKEN_LPAREN TOKEN_IDENTIFIER TOKEN_KEYWORD_IN <expr> TOKEN_RPAREN <block>
<break> →	TOKEN_KEYWORD_BREAK
<continue> →	TOKEN_KEYWORD_CONTINUE
<assignIfjIdCheck> →	<expr> | TOKEN_KEYWORD_IFJ <callIfJ>
<callIfJ> →	TOKEN_DOT <optNewlines> <callIfjCheck> TOKEN_LPAREN <varArg> TOKEN_RPAREN
<callIfjCheck> →	TOKEN_IDENTIFIER("read_str", "read_num", "write", "floor", "str", "length", "substring", "strcmp", "ord", "chr", "read_bool")
<return> →	TOKEN_KEYWORD_RETURN <returnExpr>
<returnExpr> →	<expr> | eps
<expr> →	/* precedenční analýza */

Tabulka 1: LL(1) gramatika pro jazyk IFJ25

Precedenční analýza (PSA)

Precedenční analýza výrazů byla implementována za pomocí tabulky, která je zobrazena níže. PSA umožňuje zpracování od základních matematických výrazů až po ternární operátor. PSA dále podporuje zpracovávání funkcí, které jsou součástí výrazu. Kontrola nalezení konce výrazu ve funkcích je zajištěna příznakem *flag* (bude zmíněn později), který je jako parametr předáván PSA přes překladač.

PSA začíná vytvořením zásobníků tokenů a uzlů Abstraktního syntaktického stromu (zkráceně AST). Následně je na zásobník tokenů vložen token typu `TOKEN_DOLLAR`. Následně začíná cyklus, při kterém je načten do pomocné proměnné aktuální vrchol zásobníku tokenů, na začátku to bude tedy $. V případě, že na vrcholu zásobníku je uložen již nějaký výraz, je načten token za touto položkou, v případě PSA to bude vždy $. PSA následně provede kontroly nad aktuálně načtenými tokeny, například kontrolou prvního načteného tokenu, kdy první načtený token může být buď nějaké číslo, proměnná, nebo například textový řetězec.

PSA využívá různých pomocných proměnných. Proměnná bool `firstOK` slouží k ověření správnosti prvního načteného tokenu, pokud první načtený token je validní, nastavý se do hodnoty true a kontrola již u dalších tokenů nenastává. Proměnná `checkEmptyIf` kontroluje zda výraz podmínky, nad kterým byla volána PSA není prázdný. `expectingNewLineAfterOperator` slouží ke kontrole při správnosti výrazu při vložení nového řádku za operátorem. `expectingRPAREN` slouží jako pomocná proměnná implementace zajištení kontroly ukončení výrazu pravou závorkou, využívá se například při volání PSA nad posledním parametrem funkce. `functionCall` vyvolává PSA nad zpracováním funkce jako součást výrazu, spouští se pokud PSA obdrží identifikátor a za ním následující token levé závorky. `checkIs` kontroluje správný typ klíčových slov pro datové typy (String, Num,..).

Příznak `flag` slouží jako pomocný parametr pro rozhodování PSA o ukončení výrazu, pokud obdrží hodnotu `EXPECTING_RPAREN`, ví, že výraz končí pravou závorkou, pokud `EXPECTING_ANYTHING`, je potřeba výraz zpracovávat dokud nenastane neobrdrží nevalidní znak, nebo znak nového řádku, `EXPECTING_BLOCK_END` následně značí, že PSA byla volána nad jednořádkovým voláním funkce a končí tak pravou hranatou závorkou.

	+,-	* /	<,>...	is	!=	!	&&	||	ID	$	(	)	../...	?	:
+ -	>	<	>	>	>	<	>	>	<	>	<	>	N	>	>
* /	>	>	>	>	>	<	>	>	<	>	<	>	N	N	>
<,>...	<	<	>	>	>	<	>	>	<	>	<	>	N	>	>
is	<	<	<	>	>	<	>	>	<	>	<	>	N	N	N
== !=	<	<	<	<	>	<	>	>	<	>	<	>	N	>	N
!	>	>	>	>	>	>	>	>	<	>	<	>	N	N	N
&&	<	<	<	<	<	<	>	>	<	>	<	>	N	N	N
||	<	<	<	<	<	<	<	>	<	>	<	>	N	N	N
ID	>	>	>	>	>	N	>	>	N	>	N	>	>	>	>
$	<	<	<	<	<	<	<	<	<	N	<	N	<	<	<
(	<	<	<	<	<	<	<	<	<	N	<	E	N	N	N
)	>	>	>	>	>	N	>	>	N	>	N	>	>	>	>
../...	N	N	N	N	N	<	N	N	<	N	<	>	>	N	N
?	<	<	<	N	N	N	N	N	<	N	<	N	N	N	E
:	<	<	<	N	N	N	N	N	<	>	<	N	N	N	<

Tabulka 2: Tabulka popisující implementaci PSA

Sémantická analýza

Probíhá rekurzivním postorder průchodem abstraktního syntaktického stromu pomocí funkce `recursiveNodeWalker`. Listové uzly, v závislosti na typu uzlu, vrací hodnoty určující datový typ, klíčové slovo, nebo identifikátor[1]. Kořenový uzel vrací pouze hodnotu 0 („OK“ v `SemValidationStates` číselníku), neboť nalezení chyby ukončí program.

Ostatní otcovské uzly určují svoji návratovou hodnotu v závislosti na hodnotách jejich synovských uzlů (v případě chyby dojde k ukončení programu, tudíž neexistuje chybová návratová hodnota), např.:

- Matematické operátory vrací návratovou hodnotu z číselníku `SemValidationStates` odpovídající datovému typu číselníku `SemSymbDataType`[2], pokud jsou návratové hodnoty synů validní pro daný operátor.
- Relační operátory vrací hodnotu `BOOLEAN_DATA`, pokud jsou synovské uzly číselného typu, s výjimkou operátoru rovnosti a nerovnosti, které ji vrací nezávisle na synovských uzlech.

Při nalezení identifikátoru (proměnné/funkce) je inicializován sémantický symbol pomocí funkce `initNewSymbol`, která zajistí přejmenování symbolu a uzlu ve stromě (viz tabulka 3). Nové symboly jsou přidány do vhodného rámce (reprezentovaného tabulkou symbolů), zatímco neznámé symboly do tabulky neznámých symbolů, ze které jsou odstraněny při nalezení deklarace symbolu. Pokud tato tabulka není po ukončení analýzy prázdná dojde k chybě `SEMANTICS_ERROR_UNDEF` s kódem 3.

Typ uzlu	Šablona názvu	příklad
`AST_FUNC_DEC`	*název*`$`*počet_parametrů*	`main$0`
`AST_FUNC_CALL`	*název*`$`*počet_parametrů*	`main$0`
`AST_SETTER`	*název*`$s`	`foo$s`
`AST_GETTER`	*název*`$g`	`foo$g`
`AST_VAR`	*název*`$$`*číslo_rámce*	`bar$$1`
`AST_IDENTIFIER`	viz `AST_VAR`/`AST_SETTER`/`AST_GETTER`[3]	
`AST_GLOBAL_ID`	*název*	`bar`

Tabulka 3: Formáty názvů symbolů

[1] V určitých případech upravují svoji návratovou hodnotu v závislosti na typu otcovského uzlu

[2] Převod probíhá pomocí `convSemSymbDTypeToSemValStates` a `convSemValStatesToSemSymbDType`,

[3] Konkrétní symbol, který uzel reprezentuje závisí na pozici ve stromě.

Generování kódu

Obecné principy

Kód je generován pomocí rekurzivního procházení abstraktního syntaktického stromu (`AST_STROM`) a volání specifických funkcí pro jednotlivé uzly (`AST_NODE`) na základě jejich typu (`AST_TYPE`). Číselné literály jsou při zpracování konvertovány na typ `float` (instrukce `INT2FLOATS`), čímž se redukuje počet nutných konverzí v průběhu výpočtu.

Výsledný kód u většiny operací pracuje se zásobníkem (stackem), což minimalizuje potřebu dočasných proměnných v rámci. Tento přístup je konzistentní s voláním funkcí, kde jsou parametry rovněž předávány přes datový zásobník. Typová kontrola probíhá dynamicky pomocí instrukce `TYPES`.

Správa návěští

Správa návěští (labelů) je řešena kombinací zásobníku návěští (Label Stack) a statických čítačů abychom zajistili unikátnost názvů i při vnořování bloků.

Systém funguje na následujícím principu:

- **Hierarchie a kontext:** Při vstupu do nového bloku (např. `while` cyklu nebo `if` podmínky) se na vrchol zásobníku uloží nový *prefix*. Tento prefix vzniká spojením názvu aktuální funkce, typu bloku a hodnoty unikátního čítače (např. `main_while1`).
- **Generování unikátních názvů:** Konkrétní návěští pro skoky (např. `start`, `end`, `else`) se generují připojením suffixu k aktuálnímu prefixu z vrcholu zásobníku a připojením čísla z globálního čítače. Tím vznikají unikátní identifikátory jako `main_while1_start` nebo `main_while1_if2_else`.

Práce s proměnnými a paměťové rámce

Generátor kódu využívá dvoupruchodový přístup pro zpracování těl funkcí. Před samotným generováním instrukcí proběhne projde AST podstrom dané funkce nalezne všechny deklarace lokálních proměnných a dočasné proměnné které potřebuje pro operace. Ty jsou následně hromadně definovány na začátku funkce. V rámci toho se též vygeneruji potřebné globální proměnné.

Volací konvence

Volání funkcí je realizováno plně přes datový zásobník. Umožňuje to mít v přiřazení volaní funkce.

- **Předání parametrů:** Volající funkce uloží argumenty na zásobník. Volaná funkce si je následně vyzvedne pomocí instrukcí `POPS` do lokálních proměnných v novém rámci (`CREATEFRAME`, `PUSHFRAME`).
- **Návratová hodnota:** Příkaz `return` uloží výsledek výrazu na zásobník a následně jej přesune do speciální lokální proměnné `%retval`. Před opuštěním funkce (`POPFRAME`, `RETURN`) je hodnota z `%retval` vložena zpět na datový zásobník (`PUSHS`), odkud si ji přebere volající.

Vestavěné funkce

Vestavěné funkce jsou implementovány nahrazením volání funkce za kód funkce. Parametry se předávají přes zásobník, jejich umístění zajišťuje rekurzivní průchod synovského uzlu `AST_PARAM` před samotným kódem funkce.

S cílem předejít opakovaným deklaracím jsou všechny pomocné proměnné pro vestavěné funkce deklarovány na začátku rámce.
Jejich pojmenování se řídí šablonou `LF@temp$var$`*`funkce NázevProměnné`*

Návěstí vestavěných funkcí jsou číslována pomocí interního počítadla pro jednotlivé funkce a řídí se šablonou `temp$label$`*`funkce`*`$`*`návěstí`*`$`*`číslo`*

Datové struktury

Tabulka symbolů

Pro Tabulku symbolů byla implementována samostatná datová struktura, která obsahuje pomocnou proměnnou pro uchovávání hodnoty aktuální velikost (**tableSize**), maximální kapacity tabulky (**tableCapacity**) a pole struktur položek tabulky (**items**) struktury **TableItem**. Samotná struktura položky sestává s klíče (***key***), podle kterého jsou položky vyhledávány a slouží jako vstupní parametr hashovací funkce, dále sestává se struktury **SemSymbol**, ve které jsou ukládány symboly, tedy například názvy proměnných, funkcí apod., proměnná **isEmpty** sloužící pro verifikaci položky, zda je aktuálně volná a proměnné **isDeleted**, která indikuje, zda položka je aktuálně smazána a je možné ji přeskočit. Jedná se o důležitou proměnnou, jelikož bez její implementace by mohlo dojít ke špatnému vložení vkládaného klíče.

Tabulka symbolů v základu sestáva z nastavené velikosti 20, kdy je možné vložit až 20 položek, pokud dojde k naplnění po určitou mez, ta byla stanovena na 1.2 násobek její velikost [1], dochází k realokaci celé tabulky, jedná se o dost složitý proces, kdy je potřeba alokovat nové místo na haldě a u veškerých položek vytvořit jejich nový hash a následně vložit. U staré položky dochází k uvolnění paměti položek a celé tabulky.

Hashovací funkce je implementována za pomocí algoritmu od Daniela J. Bernsteina.

Generický dynamický zásobník

Implementován pomocí maker, aby šel používat pro různé datové typy, poskytuje základní operace na inicializaci (funkce `stack##NAME##Construct`), přidání (funkce `stack##NAME##Push`), přečtení (funkce: `stack##NAME##Read`), odebrání (funkce `stack##NAME##Pop`),
deinicializaci (funkce: `stack##NAME##Destruct`) a predikát `stack##NAME##IsEmpty`. Dynamické zvětšení/zmenšení prostoru zajišťuje interní funkce `stack##NAME##Resize` a dochází k ní za podmínek popsaných v tabulce 4.

Zásobníková funkce	Podmínka	Stav zásobníku po vykonání
`stack##NAME##Push`	naplnění $> \frac{\text{kapacita}}{2}$	nová kapacita = kapacita $\cdot$ 2
`stack##NAME##Pop`	naplnění $< \frac{\text{kapacita}}{4}$	nová kapacita = $\frac{\text{kapacita}}{2}$

Tabulka 4: Změna velikosti zásobníku za určitých pravidel

Abstraktní syntaktický strom

Abstraktní syntaktický strom je v projektu implementován pomocí dvou datových struktur, které společně reprezentují strukturu programu. Tyto struktury jsou definovány v souboru `ast.h` a jejich implementace se nachází v `ast.c`

Hlavním prvkem AST je struktura `AstNode`, která představuje jeden uzel syntaktického stromu. Každý uzel obsahuje typ uzlu (`kind`), počet jeho potomků (`arity`), logickou hodnotu určující, zda se jedná o listový uzel (`isLeaf`), pole potomků (`child`) a také konkrétní hodnotu uzlu pocházející z lexikální analýzy (`value`).

Struktura `AstValue` uchovává konkrétní hodnoty listových uzlů. Vyplnění této struktury probíhá pomocí funkce `astValueFromToken()`, která extrahuje hodnotu z tokenu lexikálního analyzátoru a uloží ji do `AstValue` správným způsobem.

Uzel je vytvářen pomocí funkce `astNew`, která alokuje nový uzel, nastaví jeho vlastnosti a přiřadí mu všechny potomky.

Způsob práce v týmu

Rozdělení práce v týmu

- Vojtěch Voldřich – Sémantická analýza, implementace zásobníku, pomocné funkce, generátor kódu vestavěných funkcí
- Jakub Jambrozi – Parser, abstraktní syntaktický strom, generátor kódu
- Petr Kotek – Precedenční analýza, tabulka symbolů, generátor kódu
- Josef Pasek – Lexikální analýza, pomocné funkce, generátor kódu

Spolupráce

Jako hlavní způsob komunikace jsme zvolili Discord server s kanály pro konkrétní části projektu. Dále jsme se každý týden scházeli a společně řešili složitější záležitosti.

Vývoj probíhal v prostředí VS Code, neboť s ním všichni umíme pracovat, a nastavili jsme jej pomocí `.editorconfig`.

Na verzování jsme použili git a jako úložiště repositáře GitLab. Diagramu konečného automatu Lexikální analýzy jsme vytvořili pomocí jazyka Graphviz. Překlad na spustitelný soubor `ifj25` probíhal programem GNU MAKE.

Průběh vývoje

Během pravidelných pondělních setkání jsme řešili problémy zjištěné při vývoji, prošli další části projektu a rozdělili práci na další týden s ohledem na časové možnosti jednotlivých členů týmu.

Reference

[1] Steven. *What is a good hash table array size?* 2017. Dostupné z: `https://www.quora.com/What-is-a-good-hash-table-array-size`.